Operationalizing TRACK: Experiments, Plan Structures, and Reusable Templates for Recoverable Agentic Workflows

June 2026

Abstract

The preceding article introduced TRACK (Task Recovery And Checkpointing Knowledge) as a conceptual framework for recoverable agentic task execution. This second work studies how to transform that idea into operational practice through preliminary experiments and reusable planning templates. Two experiments compared non-TRACK (loop-based) versus TRACK-guided executions on documentary and technical-computational tasks. Results suggest that TRACK improves traceability, auditability, and artifact reuse, although failure recovery—its theoretical core advantage—was not yet demonstrated under injected fault conditions. Building on the experimental insights, a library of eleven reusable TRACK plan templates was developed, covering domains such as document summarization, Python code generation and debugging, mathematical modeling, literature review, applied research, maintenance work order analysis, FMECA generation, predictive maintenance design, and RCM analysis. Each template shares a common anatomy of task contracts, mandatory artifacts, validation gates, and recovery tasks, forming an operational pattern language for agentic workflows. The results suggest that TRACK is most valuable when tasks are long, multi-stage, evidence-dependent, and require intermediate artifacts that can be audited or reused.

Keywords: Agentic workflows, recoverable planning, checkpointing, artifact-based workflows, human-in-the-loop validation, maintenance engineering, RAMS, RCM, FMECA, predictive maintenance, AI-assisted technical work.

1 Introduction

Large Language Model (LLM)-based agents can execute long-horizon tasks by iterating through reasoning, tool use, and output generation loops. However, loop-based patterns—while flexible—tend to lose traceability, mix inferences with observations, forget intermediate decisions, or reconstruct plans only after execution. The ReAct pattern and its variants offer step-by-step reasoning but provide no built-in mechanism for persistent checkpoints, state recovery after tool failures, or mandatory validation of intermediate artifacts.

TRACK (Task Recovery And Checkpointing Knowledge) was introduced in a companion article as a conceptual framework addressing these limitations. It proposes a structured execution model:

`plan` → `task contract` → `artifact` → `validation` → `checkpoint` → `recovery` → `resume`

TRACK is grounded in five principles: (1) every task has a contract specifying inputs, outputs, and success criteria; (2) every significant output is persisted as a versioned artifact; (3)

every artifact is validated before serving as input to subsequent tasks; (4) every failure triggers diagnosis and activates a pre-designed recovery task; (5) the original plan is resumed using updated active artifact references.

The first article established the conceptual foundation and positioned TRACK within the landscape of durable execution, state machine agents, and checkpointed workflows. This second article addresses the natural next question:

> *How can TRACK be operationalized into practical planning structures that agents can use to execute complex tasks in a controlled, auditable, and recoverable way?*

The objectives are: (1) present two preliminary experiments comparing TRACK versus non-TRACK execution; (2) analyze what TRACK improved and what it could not yet demonstrate; (3) present a library of reusable TRACK plan templates as a mechanism for agent learning and standardization; (4) discuss how these plans can serve as foundations for future skills, tools, and extensions.

2 From Conceptual TRACK to Operational TRACK

The conceptual TRACK framework defines *what* a recoverable workflow should contain. Operational TRACK defines *how* that workflow is expressed, stored, executed, and validated. This transition requires three concrete artifacts:

(1) **TRACK plans**—structured documents that encode a complete workflow as a sequence of task contracts, artifact specifications, validation criteria, and recovery paths.

(2) **TRACK registers**—three CSV files (`artifact_register.csv`, `state_register.csv`, `recovery_log.csv`) that provide runtime visibility into execution progress, artifact status, and failure handling.

(3) **TRACK tools**—software utilities that automate register management, project initialization, and plan validation.

This article focuses primarily on (1) and (2), with (3) described as future work. The operational hypothesis is:

> *Reusable TRACK plans can act as operational patterns for agents, in the same way that design patterns guide software developers.*

3 Preliminary Experimental Design

Two experiments were conducted to compare loop-based (non-TRACK) execution against TRACK-guided execution. Both experiments used the same underlying LLM agent and tool set; the only difference was the execution discipline.

3.1 Experiment 1: Documentary Review Task

Task: Conduct a state-of-the-art review on recoverable agentic planning and produce a structured technical report.

Non-TRACK execution (Loop): The agent followed a loop pattern:

`search` → `read` → `reason` → `summarize` → `correct` → `deliver`

It produced a single output document (`resultado_loop.md`) containing the narrative synthesis.

TRACK execution: The agent followed a pre-defined plan with eight tasks:

`define sources` → `classify` → `summarize` → `extract concepts` → `compare approaches` → `detect gaps` → `write report` → `validate`

It produced 9 files (7 intermediate artifacts + final report + validation report).

Results: Table 1 summarizes the comparison.

Table 1: Experiment 1: Loop vs. TRACK comparison

Criterion	**Loop**	**TRACK**	**Advantage**
Completeness	8 sections, ∼2500 words	10 sections + 7 artifacts, ∼3000 words	TRACK
Clarity (reading flow)	Good, narrative	Good, modular	Loop (slight)
Source traceability	Minimal	Complete (T1–T6 traceable)	**TRACK**
Source management	25 sources listed	30 sources classified by topic, type, relevance	**TRACK**
Auditability	Low (no intermediates)	High (7 traceable artifacts)	**TRACK**
Failure recovery	N/A (no failures)	Prepared (recovery tasks defined)	TRACK (potential)
Synthesis quality	Good narrative	Structured with cross-refs	Tie

Key finding: In a controlled documentary task without injected failures, TRACK did not necessarily improve the final narrative quality, but it substantially improved process transparency, source traceability, and artifact reuse.

3.2 Experiment 2: Applied Research Task

Task: Conduct a complete applied research pipeline: literature review, method design, mathematical modeling, Python implementation, simulation with realistic data, results analysis, and technical report generation.

Domain: Joint optimization of checkpoints and preventive maintenance, inspired by the seminal work of Coffman and Gilbert on optimal checkpoint scheduling. The proposed method, ARC-PM (Adaptive Risk-Cost Checkpoint and Preventive Maintenance Optimization), was treated as an *adaptive heuristic*, not a proven optimal method.

TRACK execution: The agent followed a 19-task plan:

`scope` → `seed analysis` → `search strategy` → `literature` → `gaps` → `requirements` → `method` → `model` → `algorithm` → `code` → `execution` → `analysis` → `technical report` → `article MD` → `bibliography` → `DOCX` → LaTeX → `PDF` → `validation`

Results: Table 2 compares both executions.

Table 2: Experiment 2: Non-TRACK vs. TRACK comparison

Criterion	Non-TRACK	TRACK	Advantage
Completeness	7 docs, ∼2800 words	10 artifacts + registers + report, ∼8000 words	**TRACK**
Mathematical coherence	Correct, no derivation trace	Formal model T5, explicit transitions	**TRACK**
Code quality	Functional, well-commented	Same code, documented T6–T7	Tie
Numerical results	Identical	Identical	Tie
Report quality	Concise	Comprehensive, section-artifact links	**TRACK**
Artifact reusability	N/A (no artifacts)	T1–T9 are direct publication material	**TRACK**
Auditability	Low (must read all code)	High (each artifact validated)	**TRACK**
Recovery readiness	N/A (no failures)	Prepared (R1–R10 defined)	TRACK (potential)

Key finding: TRACK did not improve the numerical output when the same final computational model was used, but it improved the ability to audit how the model, implementation, assumptions, and results were produced. Both executions produced identical numerical results; the difference lay entirely in process governance.

4 Lessons Learned from the Experiments

Several important lessons emerged from the two experiments:

4.1 TRACK Improves Traceability, Not Magically Agent Intelligence

TRACK does not transform a poor idea into a good one. What it does is force the agent to leave a verifiable trail of: what it understood, what it researched, what it assumed, what it created, what it validated, and what it corrected. The final output quality still depends on the agent's reasoning capability and tool access; TRACK only ensures that the reasoning path is documented.

4.2 TRACK Plans Must Exist Before Execution

A critical methodological lesson is that TRACK plans cannot be reconstructed post-hoc and declared valid. For an execution to be considered TRACK-guided, the plan must exist *before* execution, and the execution must be comparable against it. Post-hoc reconstruction undermines the core promise of auditability.

4.3 Intermediate Artifacts Are More Valuable Than the Final Text

Both experiments showed that the final deliverable may appear similar between TRACK and non-TRACK execution. However, TRACK produces reusable intermediate assets: source matrices, literature summaries, mathematical model definitions, algorithm pseudocode, simulation

results, and validation reports. These artifacts can feed directly into articles, proposals, audits, or subsequent projects.

4.4 Failure Recovery Requires More Demanding Evaluation

The experiments demonstrated artifact traceability more strongly than failure recovery. Future evaluations must include injected failures—tool errors, incomplete data, broken code, inconsistent intermediate outputs, missing sources—to test whether the recovery mechanism (tasks R1–Rn) provides measurable advantage over ad-hoc recovery in loop-based execution.

5 Reusable TRACK Plans as Operational Patterns

Following the experiments, a clear practical need emerged: agents do not learn well from abstract definitions alone. They learn better from complete examples of TRACK plans applied to real tasks. This motivated the creation of a library of exemplary TRACK plan templates.

The central thesis of this section is:

> *Reusable TRACK plans can act as operational patterns for agents, just as design patterns guide software developers. Each template encodes a proven decomposition of a task category into traceable, validated, and recoverable steps.*

6 Library of TRACK Plans Developed

Eleven TRACK plan templates were developed, covering a diverse range of agentic tasks. Each is summarized below with its execution chain and key design insight.

6.1 TRACK_01: Document Summarization

Chain: `request` → `document inventory` → `extraction` → `individual summaries` → `global synthesis` → `limitations` → `final report`

Insight: The agent must not summarize directly; it must separate extraction, individual summarization, and global synthesis to preserve source-to-claim traceability.

6.2 TRACK_02: Python Code Generation

Chain: `request` → `functional spec` → `technical spec` → `pseudocode` → `code` → `tests` → `execution` → `correction` → `delivery`

Insight: Code must not be written before the functional specification, technical design, pseudocode, and test plan exist and are validated.

6.3 TRACK_03: Python Code Debugging

Chain: `problem` → `reproduce error` → `diagnose` → `fix strategy` → `fix` → `test` → `validate` → `deliver`

Insight: No fix should be applied without first reproducing the error and diagnosing root cause. Regression tests must accompany every fix.

6.4 TRACK_04: Literature Review

Chain: `research question` → `search strategy` → `candidate sources` → `selection` → `classification` → `summaries` → `comparative matrix` → `gaps` → `synthesis`

Insight: No conclusion should be drawn from unregistered sources. Every claim must link to a classified, summarized, and evaluated source.

6.5 TRACK_05: Mathematical Model Creation

Chain: `problem` → `scope` → `assumptions` → `variables` → `parameters` → `relationships` → `objective function` → `constraints` → `validation` → `interpretation`

Insight: No symbol may appear without definition. Every assumption must be explicit. Units must be coherent.

6.6 TRACK_06: Complete Applied Research

Chain: `problem` → `literature` → `gaps` → `requirements` → `method` → `model` → `algorithm` → `code` → `execution` → `results` → `report` → `article` → `DOCX` → LaTeX → `PDF`

Insight: TRACK can govern not only technical investigation, but also the complete editorial chain from raw results to publication-ready formats.

6.7 TRACK_07: Maintenance Work Order Analysis

Chain: `raw data` → `field inventory` → `data quality` → `cleaning` → `OT classification` → `failure modes` → `functional events` → `RAM KPIs` → `report`

Insight: A corrective work order is not automatically a functional failure event. The transformation from OT to event must be explicit and justified.

6.8 TRACK_08: FMECA Generation from Documents and OTs

Chain: `system` → `sources` → `functions` → `functional failures` → `failure modes` → `causes/mechanisms` → `effects` → `controls` → `criticality` → `FMECA`

Insight: Every row in the FMECA table must have a source, a confidence level, and an evidence type (observed, documented, or inferred).

6.9 TRACK_09: Maintenance Planning

Chain: `assets` → `criticality` → `failures` → `existing tasks` → `gaps` → `proposed tasks` → `frequencies` → `resources` → `plan`

Insight: Every maintenance task must be linked to a failure mode, a source, or a technical justification. No task should float without rationale.

6.10 TRACK_10: Predictive Maintenance Design

Chain: `critical assets` → `predictable failure modes` → `physical variables` → `sensors` → `data quality` → `rules/models` → `results` → `dashboard`

Insight: Not every critical failure is predictable. Sensor proposals must be physically justified. Advanced ML should not be used without sufficient data volume and quality.

6.11 TRACK_11: RCM Analysis

Chain: `system` → `operational context` → `functions` → `functional failures` → `failure modes` → `effects` → `consequences` → `existing tasks` → `decision logic` → `task selection` → `frequencies` → `RCM plan`

Insight: In RCM, the maintenance task must derive from the consequence category (hidden, safety, environmental, operational, non-operational), not solely from the failure mode.

7 Common Anatomy of TRACK Plans

Despite their domain diversity, all eleven TRACK plans share a common architecture:

1. **Plan name**—a clear identifier (e.g., `TRACK_Analisis_RCM`).
2. **Objective**—what the plan resolves and with what controls.
3. **Intended use**—when this plan should be selected.
4. **Inputs**—required and optional data, documents, and constraints.
5. **Final outputs**—deliverables the plan must produce.
6. **Folder structure**—recommended project directory layout.
7. **Main artifacts**—a table mapping artifact IDs to descriptions.
8. **Mandatory registers**—the three CSV control files.
9. **TRACK tasks**—each with objective, inputs, actions, outputs, success criteria, validation, next task, and recovery.
10. **Recovery tasks**—pre-designed recovery paths for each failure mode.
11. **Special rules**—domain-specific constraints and prohibitions.
12. **Final success criteria**—what must exist and be verified for the plan to be complete.
13. **Complete plan chain**—a visual summary of the task sequence.
14. **Difference from non-TRACK**—explicit contrast with loop-based execution.

This common anatomy enables: agent training through examples, automated validation of plan completeness, cross-execution comparison, third-party auditing, and plan reuse across projects.

8 TRACK Registers as Execution Control

The three mandatory CSV registers serve distinct control functions:

Table 3: TRACK registers and their control functions

Register	Columns	Question Answered
`artifact_register.csv`	`artifact_id, task_id, version, path, status, created_by, replaces, used_by`	What was produced and where is it?
`state_register.csv`	`step, current_task, status, active_artifact_refs, next_task, notes`	Where is the agent in the plan?
`recovery_log.csv`	`failure_id, failed_task, failure_type, diagnosis, recovery_task, result, updated_reference`	What failed and how was it corrected?

The `artifact_register` enables version tracking and dependency management. When an artifact is corrected (e.g., a mathematical model updated after validation failure), the new version supersedes the old one, and all downstream tasks reference the active version.

The `state_register` provides a linear execution log. At any point, a human auditor can determine exactly which task is active, what artifacts are in scope, and what the next planned step is.

The `recovery_log` captures controlled failure handling. Instead of ad-hoc retries, each failure is diagnosed, classified (missing_data, invalid_output, tool_error, missing_source, quality_insufficient, ambiguity), and mapped to a pre-designed recovery task.

9 Comparison: TRACK Plan vs. Traditional Prompt

Table 4 contrasts a traditional prompt-based execution with a TRACK plan-guided execution across eight dimensions.

Table 4: TRACK plan vs. traditional prompt

Dimension	**Traditional Prompt**	**TRACK Plan**
State visibility	Implicit (in conversation)	Explicit (state_register.csv)
Artifacts	Optional, ephemeral	Mandatory, versioned, persisted
Validation	Informal, post-hoc	Per task, criteria-driven
Recovery	Ad-hoc, improvised	Pre-designed recovery tasks
Traceability	Weak (output-only)	Strong (artifact chain)
Reusability	Low (monolithic output)	High (modular artifacts)
Auditability	Difficult	Systematic
Agent learning	From scratch each time	Template instantiation

A traditional prompt says: *"Write a report on X."* A TRACK plan says: *interpret X, research X, register sources, extract information, create artifacts, validate each, correct if needed, deliver, declare limitations.*

10 Discussion

10.1 TRACK as a Governance Pattern, Not an Execution Engine

TRACK does not replace LangGraph, AutoGen, CrewAI, OpenAI Agents SDK, or similar agent frameworks. It can be implemented *on top of* them as a planning and control structure. The execution engine remains the underlying agent runtime; TRACK provides the governance layer that ensures recoverability and auditability.

10.2 TRACK as a Common Language Between Human and Agent

A TRACK plan serves as a shared contract: the human reviewer knows *before execution* what the agent will do, what it will produce, and how it will handle failures. This transforms the agent from an opaque executor into a transparent collaborator.

10.3 TRACK as a Foundation for Skills, Tools, and Extensions

The eleven plan templates can be directly converted into:

- **Planning skills**—agents that generate TRACK plans from user requests.
- **Validation extensions**—automated checkers that verify artifact completeness.
- **Register tools**—utilities that manage `artifact_register`, `state_register`, and `recovery_log`.
- **Plan validators**—automated checks that a TRACK plan contains all required sections.
- **Execution auditors**—post-execution analysis that compares actual execution against the TRACK plan.

A concrete implementation of TRACK tools (for the Pi coding agent platform) was realized as a proof of concept, including five tools: `track_init` (project initialization), `track_artifact` (artifact registration), `track_state` (state updates), `track_recover` (recovery logging), and `track_validate` (plan validation).

10.4 TRACK as an Example-Based Learning Mechanism

Agents learn more effectively from concrete plan examples than from abstract definitions. The library of eleven plans serves a dual purpose: operational execution templates and training examples that demonstrate the TRACK pattern across diverse domains.

11 Limitations

This work has several important limitations:

(1) **Preliminary experiments.** Only two experiments were conducted, using a single agent configuration. Generalizability across agents, models, and task types is not established.

(2) **No multi-agent testing.** TRACK was tested with a single-agent setup. Multi-agent coordination introduces additional complexity not addressed here.

(3) **No statistical comparisons.** Results are qualitative comparisons of single executions, not statistical analyses over multiple runs.

(4) **Failure recovery not demonstrated.** No failures were injected in either experiment, so the core TRACK advantage—controlled recovery—remains theoretically argued rather than empirically demonstrated.

(5) **Manually designed plans.** All eleven templates were designed manually. Automated plan generation from task descriptions is future work.

(6) **Execution quality is agent-dependent.** TRACK plans provide structure but cannot compensate for weak reasoning, poor tool use, or insufficient domain knowledge of the underlying agent.

(7) **Human validation still required.** For safety-critical, regulatory, or high-risk domains, TRACK artifacts are drafts pending expert review.

(8) **Artifact existence $\neq$ reasoning correctness.** A validated artifact means the agent followed the plan, not that its reasoning was sound.

(9) **Overhead for simple tasks.** TRACK adds process overhead (planning time, artifact management, register maintenance) that may not be justified for simple or exploratory tasks.

12 Future Work

Several lines of future work emerge from this study:

1. Execute TRACK plans on real industrial cases in RAMS, maintenance, and FMECA domains.
2. Measure execution time, output quality, error rates, and recovery effectiveness quantitatively.
3. Inject controlled failures (missing data, tool errors, code bugs, inconsistent outputs) to test recovery mechanisms.
4. Create an automated TRACK plan validator that checks structural completeness.
5. Develop a skill that enables agents to generate TRACK plans dynamically from user requests.
6. Implement tools that automate the three CSV registers (as prototyped for the Pi platform).
7. Apply TRACK to real RAMS projects with regulatory traceability requirements.
8. Apply TRACK to technical software generation in engineering contexts.
9. Evaluate impact on productivity, documentation quality, and stakeholder confidence.

13 Conclusions

This work demonstrates that TRACK can transition from a conceptual framework to operational practice through structured plans, controlled registers, and reusable templates. The key findings are:

(1) Preliminary experiments suggest that TRACK improves traceability, auditability, and artifact reuse in complex agentic tasks.

(2) The clearest advantage appears in long, multi-stage, evidence-dependent tasks where intermediate artifacts have value beyond the final deliverable.

(3) The eleven TRACK plan templates function as reusable operational patterns, covering a wide range of technical and engineering domains.

(4) The common anatomy shared by all plans facilitates agent learning, plan validation, execution comparison, and auditing.

(5) Failure recovery—TRACK's theoretical core advantage—requires experiments with injected failures for empirical demonstration.

(6) TRACK is especially promising in domains where traceability is critical: RAMS, maintenance engineering, RCM, FMECA, predictive maintenance, applied research, and technical software generation.

TRACK does not merely ask agents to work harder; it asks them to work in a way that can be inspected, resumed, corrected, and trusted.

References

[1] E. G. Coffman Jr. and E. N. Gilbert. Optimal strategies for scheduling checkpoints and preventive maintenance. *IEEE Transactions on Reliability*, 39(1):9–18, 1990.

[2] TRACK Methodology. TRACK: Task Recovery And Checkpointing Knowledge — A conceptual framework for recoverable agentic task execution. Companion article, 2026.

[3] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *arXiv:2210.03629*, 2022.

[4] LangChain. LangGraph: Persistence and checkpointing documentation. `https://langchain-ai.github.io/langgraph/`, 2026.

[5] Temporal Technologies. Durable execution for long-running workflows. `https://temporal.io/`, 2026.

[6] Inngest. Durable execution: Key to harnessing AI agents. Technical article, 2025.

[7] Zylos Research. AI agent workflow checkpointing and resumability. Technical article, March 2026.

[8] Zylos Research. Durable execution for AI agent runtimes. Technical article, April 2026.

[9] Agents' Codex. AI agent state machines: Designing persistent workflows. Technical article, April 2026.

[10] AgentMarketCap. State machine vs. ReAct: Production reliability. Technical article, April 2026.

[11] Diagrid (Temporal). Why checkpoints aren't durable execution. Technical article, 2026.

[12] Anonymous. Intermediate artifacts as first-class citizens in agentic workflows. *arXiv*, May 2026.

[13] A. Agrawal. Self-healing agentic orchestrators for reliable tool-augmented LLM systems. *arXiv*, June 2026.

[14] Anonymous. DART: Semantic recoverability for structured tool agents. *arXiv*, May 2026.

[15] Anonymous. ADEMA: Knowledge-state orchestration for long-horizon synthesis. *arXiv*, April 2026.

[16] ACL ARR. When tools fail: Benchmarking dynamic replanning and anomaly recovery. *arXiv*, June 2026.

[17] Anonymous. From agent traces to trust: Evidence tracing survey. *arXiv*, June 2026.

[18] Microsoft. Microsoft Agent Framework — Workflow checkpoints. Official documentation, 2026.

[19] Effloow. AI agent frameworks compared 2026. Comparative article, 2026.

[20] R. Walker. Agent frameworks compared. Research article, 2026.

A Example TRACK Plan Skeleton

Below is the canonical skeleton shared by all TRACK plans:

```
# TRACK_<name>.md

## Name of the plan
## Objective
## Intended use
## TRACK principle applied
```

```
## 1. General inputs
## 2. Final outputs
## 3. Recommended folder structure
## 4. Main artifacts (table)
## 5. Mandatory TRACK registers
## 6. TRACK tasks (T1...Tn)
## 7. Recovery tasks (R1...Rn)
## 8. Special rules
## 9. Final success criteria
## 10. Complete plan chain
## 11. Usage example
## 12. Difference vs. No-TRACK execution
```

B TRACK Plan Evaluation Checklist

Table 5 provides a checklist for evaluating the quality of any TRACK plan.

Table 5: TRACK plan evaluation checklist

Criterion	**Question**
Scope	Does the plan clearly define what will and will not be done?
Inputs	Does it identify data, sources, and documents needed?
Research	Does it include a search/study phase if context is missing?
Artifacts	Does each task produce a verifiable artifact?
Validation	Does each task have explicit success criteria?
Recovery	Are explicit recovery tasks defined?
Registers	Does it include artifact/state/recovery logs?
Traceability	Does the final output derive from prior artifacts?
Limitations	Does it declare uncertainties and assumptions?
Executability	Can it be followed by a real agent with available tools?